

SVEUČILIŠTE U ZAGREBU
FAKULTET ELEKTROTEHNIKE I RAČUNARSTVA

Projekt

**PRIMJENA VACUUM FILTERA PRI
TRAŽENJU SLUČAJNIH PODNIZOVA U E.
COLI GENOMU**

Matheo Kesar, Nika Valić

Voditeljica:

prof. dr. sc. Mirjana Domazet-Lošo

Zagreb, svibanj, 2026

Sadržaj

1. Uvod
2. Implementacija
 - 2.1. Opis rada Vacuum Filtera
 - 2.2. Funkcije umetanja, brisanja i pretraživanja
 - 2.2.1. Funkcija umetanja
 - 2.2.2. Funkcija brisanja
 - 2.2.3. Funkcija pretraživanja
 - 2.3. Parsiranje E. coli genoma
 - 2.4. Rezultati
3. Zaključak
4. Literatura

1. Uvod

Često je u praksi potrebno ispitati nalazi li se neki specifični element u nekom velikom skupu elemenata (kao što je baza podataka, mrežni firewall ili genom organizma), ali svako takvo pretraživanje (ako je riječ o nesortiranom skupu) odvija se u linearnom vremenu što značajno usporava procese koji često trebaju izvoditi takvu radnju. Kako bismo riješili taj problem prva pomisao bila bi implementirati hash tablicu. Ona pruža umetanje, brisanje i pretraživanje u $O(1)$, konstantnom vremenu. Drugi problem koji se može javiti je da sama struktura hash tablice može biti dosta velika i zato pronalazimo rješenje u tzv.

probabilističkoj strukturi podataka. One pružaju mogućnost upita o približnoj pripadnosti elementa. To znači da ako se element ne nalazi u skupu podataka, algoritam će sigurno vratiti da se element ne nalazi, a ako se nalazi u skupu podataka, algoritam će vratiti da se nalazi s nekom vjerojatnošću pogreške koja se može prilagoditi prilikom implementacije. Najčešći primjeri ovakvih struktura su Bloom filteri. Kao najkorištenija podatkovna struktura oni omogućavaju mali udio lažno pozitivnih rezultata uz dobru prostornu učinkovitost, no ne dopuštaju brisanje elemenata skupa. Kako bi se unaprijedio Bloom filter, predstavljen je Cuckoo filter koji omogućava brisanje, no zakazuje kada veličine tablica u prostoru nisu potencija broja 2 te kada se ubacuju duplikati u skup.

Vacuum filter je modifikacija Cuckoo filtera, a prednost mu je učinkovito korištenje memorije te provedba vrlo malog broja lažno pozitivnih rezultata. Koristi do 25 % manje prostora od ostalih struktura te pruža veću propusnost kod umetanja i pretraživanja.

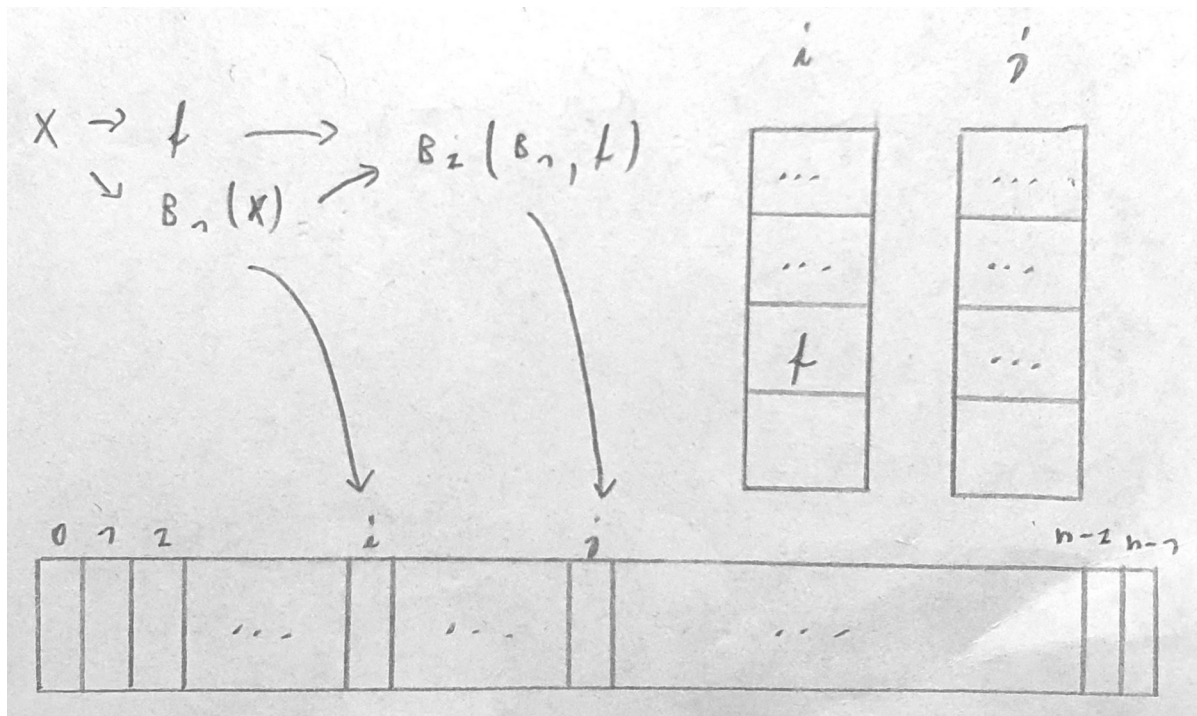
Problem za kojeg se može koristiti vacuum filter je pretraživanje slučajnih podnizova *E. coli* genoma. Korištenje vacuum filtera poželjno je za veliku podatkovnu strukturu poput genoma bakterije jer omogućuje da se više podataka nalazi u brznoj memoriji što je ključno za performanse. Također se koristi IUPR (engl. *Instant Updates and Periodical Reconstructions*) što omogućuje da učinkovito obrađuje umetnute duplikate i brisanje tako da rekonstruira filtere u pozadini. U ovom radu implementirane su funkcije umetanja, brisanja i pretraživanja.

2. Implementacija

U ovoj će se cjelini detaljnije opisati kako funkcionira primjena vacuum filtera na *E. coli* genomu za pretraživanje slučajnih podnizova veličine k .

2.1. Opis rada Vacuum Filtera

Vacuum filter je tablična AMQ (engl. *approximate membership query*) podatkovna struktura koja pohranjuje otiske prstiju (engl. *fingerprints*) elemenata kako bi odredili pripadnost s visokom učinkovitošću i brzom memorijom. Ona radi tako da umeće otiske prstiju u tablicu sačinjenu od kutija (*buckets*) od kojih svaki sadrži 4 mjesta (*slots*).



Slika 1: Dijagram tablice s dva bucketa kojima pripada fingerprint f podatka x

Kada želimo ubaciti ili tražiti neki element, prvo će hash funkcija stvoriti reprezentaciju podatka, tj. otisak prsta. Za taj fingerprint će onda hash funkcija pronaći njegov bucket. Koristi mapping funkciju (1)

$$B_1(x) = \frac{\text{hash}(x)}{2^{32}} \cdot n \quad (1)$$

n - veličina tablice (ukupan broj bucketa)

hash – hash funkcija koja vraća 32 bitni hash

Kako bi se održala razina lokalnosti (zbog učinkovitijeg smještanja u *CPU cache*), poželjno je gledati na cijelu tablicu u komadima (*chunkovima*). Drugi bucket kojem element može pripasti treba biti u istom chunku. Za svaki podatak x računa se indeks alternativnog bucketa (2):

$$B_2(x) = Alt(B_1(x), f) = B_1(x) \oplus (hash(f) \bmod L) \quad (2)$$

L - broj uzastopnih bucketa u jednom chunku (veličina chunka)

f – fingerprint podatka x

Ova formula omogućava filteru da pronađe drugi bucket koristeći samo broj prvog bucketa i fingerprint elementa. Važno svojstvo ovih formula je da su reverzibilne:

$$Alt(B_1(x), f) = B_2$$

$$Alt(B_2(x), f) = B_1$$

Vacuum filter uvodi koncept alternativnih raspona kako bi se uravnotežio kompromis faktora opterećenja i lokalnosti podataka. Umjesto da koristi jednaku veličinu chunkova, koristi se nekoliko različitih veličina, a odgovarajuća veličina odabire se prilikom računanja pozicija bucketa. Faktor opterećenja je omjer popunjenih mjesta u tablici i svih mjesta. Vacuum filter koristi 4 alternativna raspona. Manji raspon osigurava najbolju lokalnosti zbog blizine bucketa. Veći će raspon osigurati bolji raspon opterećenja jer će elementi biti bolje raspodijeljeni po tablici. Veličine rasporeda se računaju prvo algoritmom LoadFactorTest koji provjerava je li AR (*alternate range*) moguć te RangeSelection koji nalazi minimalni AR, kako bi faktor opterećenja bio oko 95 %.

```

bool LoadFactorTest(int items, float a, float r, int L, int slots){
    // aproximation of the load
    // items      - maximum num of items
    // a          - target load factor (around 96%)
    // r          - multiplier for a - for a more certain prediction (around 95%)
    // L          - alternate range - number of sequential buckets
    // slots      - number of slots per bucket

    // num of buckets (multiple of L)
    int m = ceil(items / a / slots / L) * L;
    // target number of entries
    int N = slots * m * r * a;
    // number of chunks
    int c = m / L;
    // min chunk capacity
    float P = 0.97 * slots * L;
    // max load estimator - "Balls into Bins" problem...
    float D = (double)N/c + 3.0/2*sqrt(2.0*N/c*log(c));

    return (D < P) ? true : false;
}

int RangeSelection(int items, float a, float r, int slots){
    int L = 1;
    while (LoadFactorTest(items, a, r, L, slots) == false) L *= 2;
    return L;
}

```

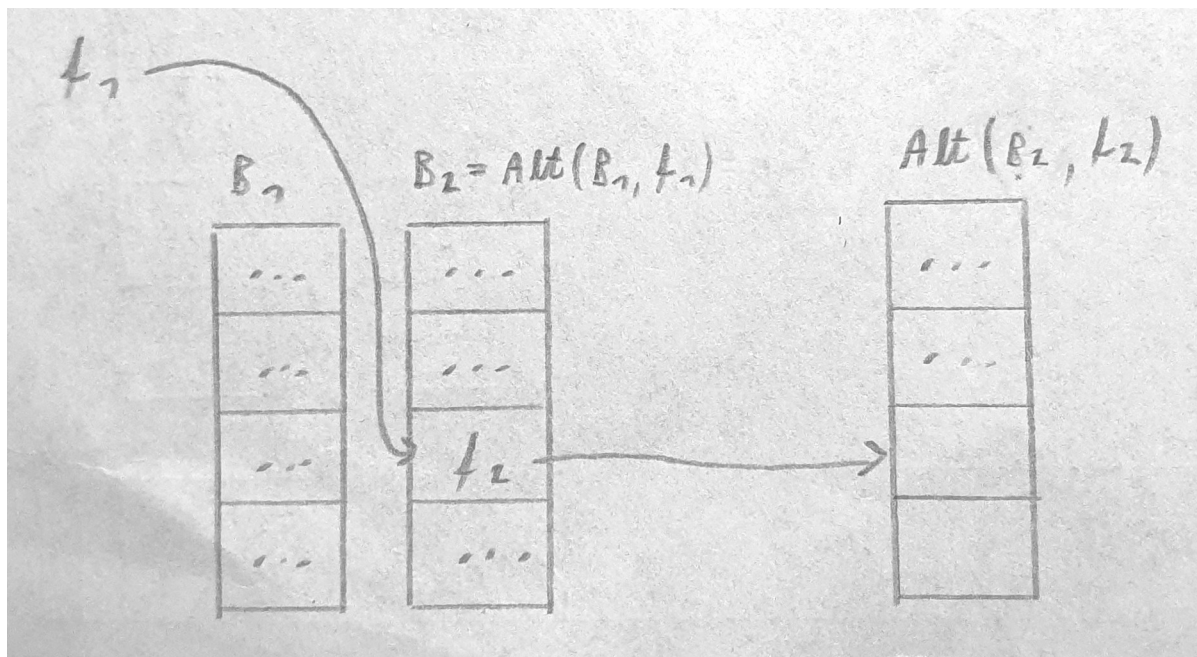
Slika 2: Kod za algoritme LoadFactorTest i RangeSelection

U ovoj implementaciji nije implementirano polu-sortiranje te se otisci prstiju pohranjuju direktno u tablicu. Implementiranje polu-sortiranih elemenata smanjio bi otiske prstiju za par bitova (na 13 umjesto 16 kako je ovdje implementirano). To je povećanje memorijskog zauzeća najviše za 23% pa smo odlučili izostaviti tu implementaciju.

2.2. Funkcije umetanja, brisanja i pretraživanja

2.2.1. Funkcija umetanja

Funkcija umetanja računa otisak prsta podatka pomoću hash funkcije te određuje 2 kandidatska bucketa, B_1 i B_2 . Ako jedan od bucketa ima slobodno mjesto, fingerprint se tamo upisuje. Ako su oba kandidatska bucketa puna, onda se provodi proces zamjene. Algoritam će izabrati jedan od fingerprinta iz tih bucketa i pogledat će može li ga prebaciti u njegov alternativni bucket. Ako uspije, onda ga premješta tamo, a na njegovo mjesto stavlja fingerprint novog podatka. Ako nema slobodnih mjesta, nasumično se odabere jedan fingerprint, zamjenjuje se s novim elementom i traži se novo mjesto za izbačeni otisak prsta u alternativnom bucketu. Ovaj postupak će se ponavljati dok ne pronađe slobodno mjesto ili dok se ne dosegne *maxEvict*. Ako dođe do maksimalnog broja izbacivanja i umetanje nije uspjelo, sve se poništi.



Slika 3: Prikaz ubacivanja znaka f_1 i prebacivanje f_2 u njegov alternativni bucket

Algorithm 5: Vacuum Filter : Insert(x)

```
 $f = H'(x)$  //  $H'$  is the fingerprint function;  
 $B_1 = H(x)$ ,  $B_2 = \text{Alt}(B_1, f)$ ;  
if  $B_1$  or  $B_2$  has an empty slot then  
    put  $f$  into the empty slot ;  
    return Success;  
end  
Randomly select a bucket  $B$  from  $B_1$  and  $B_2$  ;  
for  $i = 0$ ;  $i < \text{MaxEvicts}$ ;  $i++$  do  
    // Extend Search Scope  
    foreach fingerprint  $f'$  in  $B$  do  
        if Bucket  $\text{Alt}(B, f')$  has an empty slot then  
            put  $f$  to the original slot of  $f'$  ;  
            put  $f'$  to the empty slot ;  
            return Success ;  
        end  
    end  
    Randomly select a slot  $s$  from bucket  $B$  ;  
    Swap  $f$  and the fingerprint stored in the slot  $s$  ;  
     $B = \text{Alt}(B, f)$  ;  
end  
return Fail ;
```

Slika 4: Algoritam umetanja[1]

2.2.2. Funkcija brisanja

Funkcija brisanja, isto kao i funkcija umetanja, za element prvo računa otisak prsta i kandidatske buckete, B1 i B2. Krenu se pretraživati sva mjesta u B1 i B2 dok se ne pronade jednaka vrijednost otiska prsta kao i za željeni element. Ako se nađe otisak, na to mjesto se upisuje 0 (prazno mjesto).

```
template <typename T>
bool VacuumFilter<T>::del(uint64_t x)
{
    /*
    f = fingerprint(x)
    b1 = hash(x)
    b2 = alt(b1, f)

    if f in b1 or b2:
        remove f from one of them
        return true
    return false
    */

    T f = fingerprint(x);
    unsigned int b1 = pos_hash(x);
    unsigned int b2 = alt(b1, f);

    for (int i = 0; i < this->m; i++){
        if (table[b1*this->m + i] == f){
            table[b1*this->m + i] = 0;
            this->filled_cells--;
            return true;
        }
        if (table[b2*this->m + i] == f){
            table[b2*this->m + i] = 0;
            this->filled_cells--;
            return true;
        }
    }
    return false;
}
```

Slika 5: Kod za funkciju brisanja

2.2.3. Funkcija pretraživanja

Kod pretraživanja elementa trebaju se izračunati kandidatski bucketi B1 i B2. Ako se otisak prsta traženog elementa pronađe u jednom od 2 bucketa, pretraživanje je uspješno.

```
template <typename T>
bool VacuumFilter<T>::lookup(uint64_t x)
{
    /*
    f = fingerprint(x)
    b1 = hash(x)
    b2 = alt(b1, f)

    if f in b1 or b2:
        return true
    return false
    */

    T f = fingerprint(x);
    unsigned int b1 = pos_hash(x);
    unsigned int b2 = alt(b1, f);

    for (int i = 0; i < this->m; i++){
        if (table[b1*this->m + i] == f){
            return true;
        }
        if (table[b2*this->m + i] == f){
            return true;
        }
    }
    return false;
}
```

Slika 6: Kod za funkciju pretraživanja

2.3. Parsiranje *E. coli* genoma

E. coli genom učitava se iz FNA datoteke koja se sastoji od zaglavlja i slijeda nukleotida.

Kako bi se izvukli podnizovi iz podataka prolazimo kliznim prozorom veličine k kroz cijelu sekvencu. Dobivene podnizove provodimo kroz hash funkciju MurMurHash64. Vrijednosti koje vraća hash funkcija ubacujemo u filter.

Kako bi se testirali lažno pozitivni rezultati, generiraju se nasumični hashevi koji nisu u skupu hasheva k -mera genoma. Brojevi koji se generiraju trebali bi dati negativne rezultate, a svaki pozitivan se broji kao lažno pozitivan. Za generiranje tih brojeva koristi se Mersenne Twister, mt19937_64.

Uz testiranje genoma *E. coli*, koriste se i “umjetno” generirani podatci.

```
vector<uint64_t> extract_kmer(const string& file_path, int k){
    ifstream file(file_path);
    string seq = "";
    string line;

    if(!file.is_open()){
        cout << "Error opening file.";
        return {};
    }

    while(getline(file, line)){
        if(line.empty()){
            continue;
        }
        if(line[0] == '>'){
            continue; // Skipping the header
        }
        seq += line;
    }
    file.close();

    vector<uint64_t> kmers;
    for(int i = 0; i <= (int)seq.size() - k; i++){
        uint64_t kmer_hash = hash_kmer(seq, i, k);
        if(kmer_hash != 0){
            kmers.push_back(kmer_hash);
        }
    }

    return kmers;
}
```

Slika 7: kod za ekstrakciju k -mera

```

vector<uint64_t> generate_random_kmer(const vector<uint64_t>& existing_kmer, int num){
    unordered_set<uint64_t> existing_set(existing_kmer.begin(), existing_kmer.end()); // All values in our existing k-mers
    cout << "Repeated kmer hashes: " << existing_kmer.size() - existing_set.size() << endl;
    vector<uint64_t> random_kmers;
    int seed = random_device{}();
    mt19937_64 rnd(seed); // Pseudo-random generator of 64-bit numbers

    while((int)random_kmers.size() < num){
        uint64_t val = rnd();
        if(val == 0){
            continue; // 0s are reserved for empty slots
        }

        if(existing_set.find(val) == existing_set.end()){
            random_kmers.push_back(val); // We're only adding values that aren't the the existing k-mers
        }
    }
    return random_kmers;
}

```

Slika 8: Generiranje nasumičnih k-mera

```

vector<uint64_t> artificial_data(int n){
    vector<uint64_t> data;
    int seed = random_device{}();
    mt19937_64 rnd(seed);

    while((int)data.size() < n){
        uint64_t val = rnd();
        if (val != 0){
            data.push_back(val);
        }
    }
    return data;
}

```

Slika 9: Generiranje “umjetnih” podataka za testiranje VF-a

2.4. Rezultati

Rezultati implementacije su spremljeni u results.csv gdje su zabilježeni podaci poput tip podataka, k, udio lažno pozitivnih rezultata, propusnost (brzina) umetanja i pretraživanja u *mops*, faktor opterećenja i efektivni bitovi po elementu.

data_type	k	false_positive_rate	insert_throughput [MOPS]	lookup_throughput [MOPS]	load_factor	bits_per_item
ecoli	10	2.7e-05	0.237869	61.7565	0.621355	25.7502
ecoli	20	0.0001	19.5457	62.7834	0.943671	16.9551
ecoli	50	0.000106	28.3579	69.0854	0.944203	16.9455
ecoli	100	0.000116	29.8585	73.6447	0.944231	16.9451
ecoli	200	0.000111	30.2997	73.3334	0.944231	16.9451
artificial	0	0.000125	39.2584	75.2867	0.897576	17.826

Kroz testiranje vacuum filtera pokazalo se da je faktor opterećenja 97% s vjerojatnošću od 99%, stoga se faktor opterećenja fiksira na 95% i odabire se AR vrijednost takva da broj elemenata nikad neće prijeći 97% kapaciteta chunka. Naši rezultati pokazuju da se faktor opterećenja takve implementacije kreće oko 95% za $k \geq 20$ što je bilo i za očekivati.

Udio lažnih podataka je iznimno mali: [1]

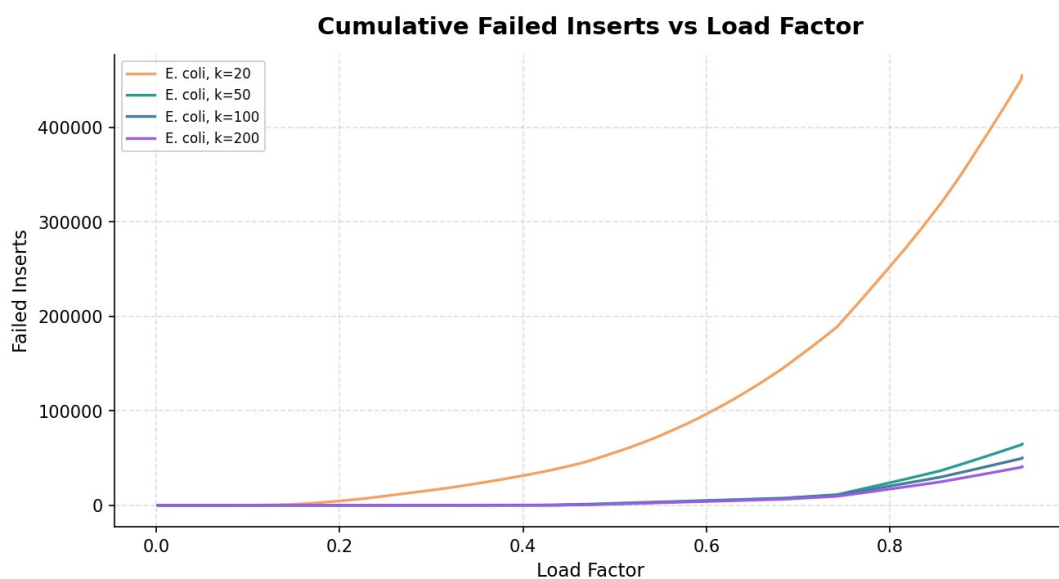
$$\epsilon = 1 - (1 - 1/2^l)^{2b\alpha}$$

l - duljina otiska prsta, koristimo 16-bitne otiske

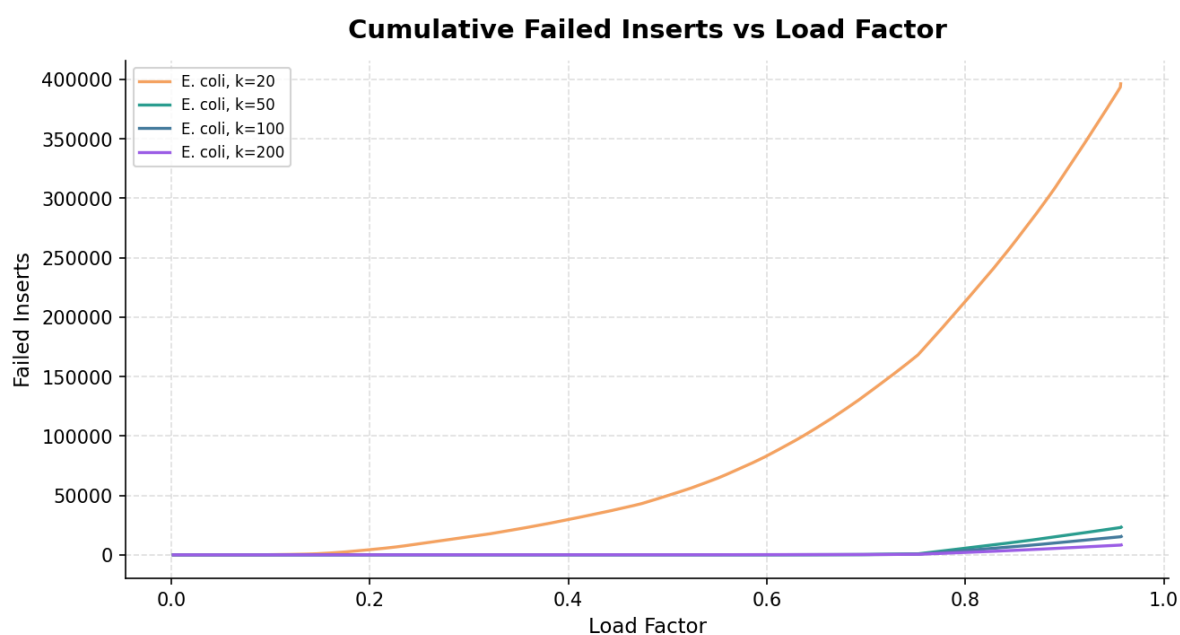
b - broj mjesta u bucketu

α - faktor opterećenja

Vacuum filter je davao loše rezultate za $k = 10$ zbog puno duplikata k-mera te jednakih hasheva koji se pokušavaju smjestiti u iste buckete pa se brzo napune.

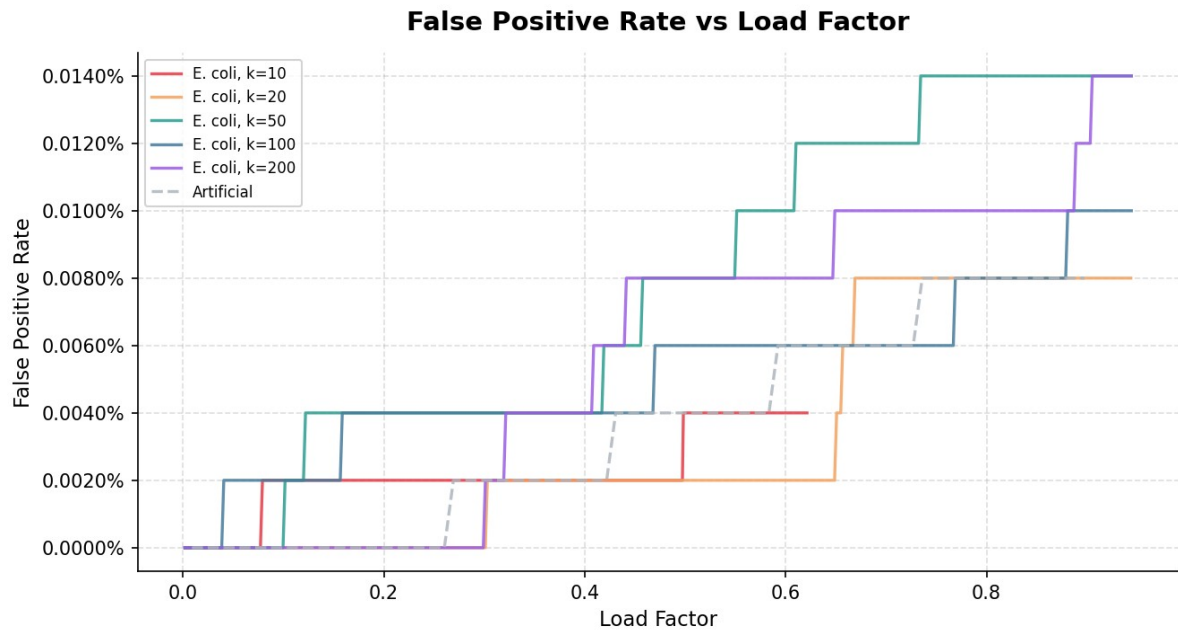


Slika 10: Neuspješna umjetanja naspram faktor opterećenja, naša implementacija

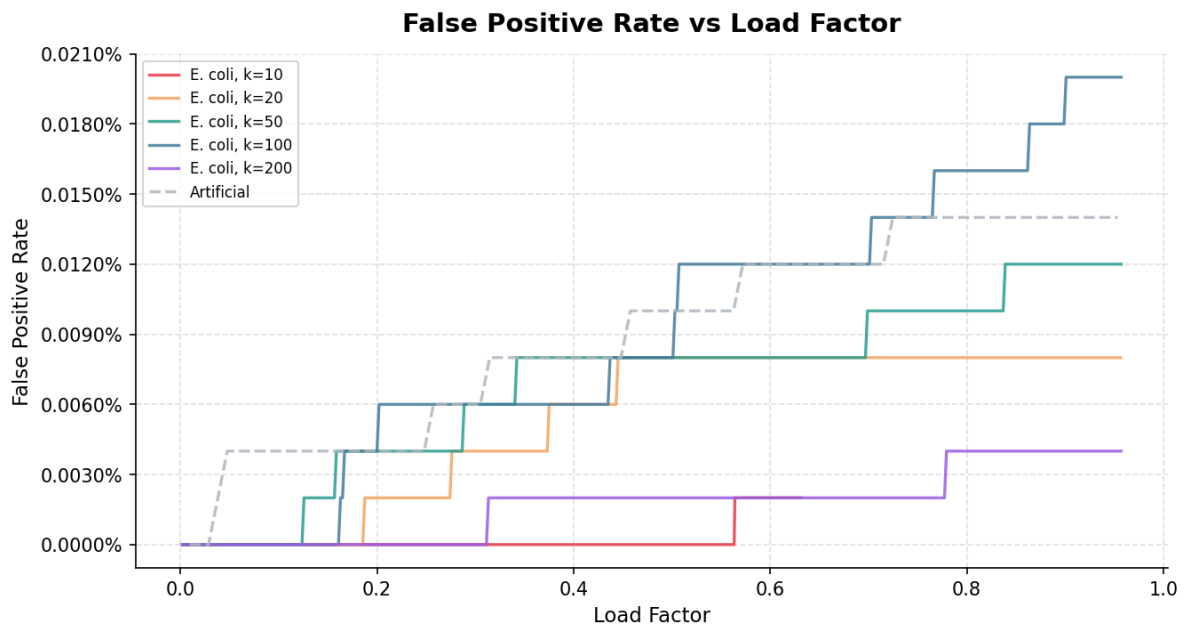


Slika 11: Neuspješna umetanja naspram faktor opterećenja, originalna implementacija

Za $k = 20$ se obje implementacije ponašaju slično, neuspješno umetanje kreće oko 0.2 faktora opterećenja i povećava se s faktorom. Za k od 50 do 200 naša implementacija ima više neuspjeha.

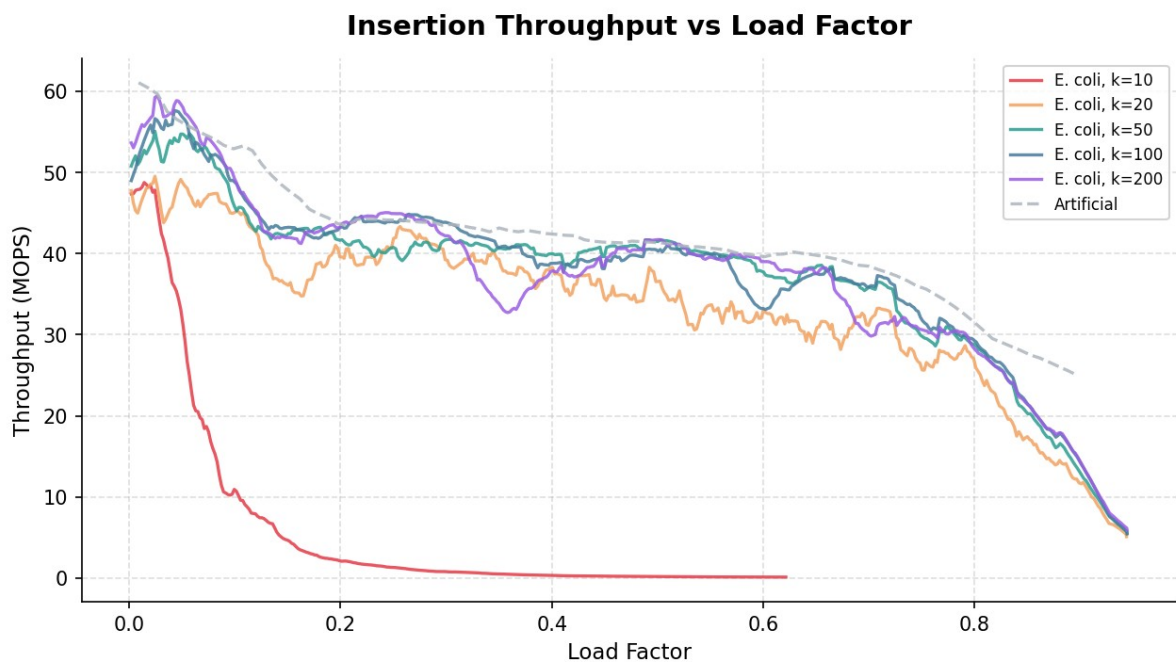


Slika 12: Udio lažno pozitivnih rezultata naspram faktora opterećenja, naša implementacija

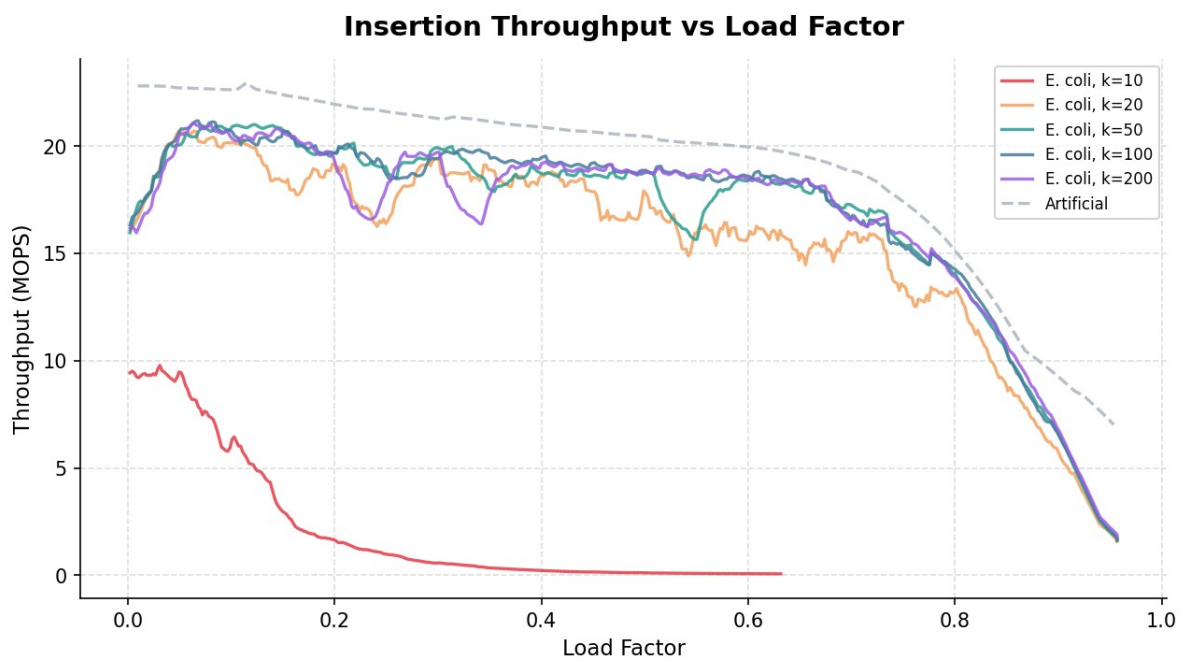


Slika 13: Udio lažno pozitivnih rezultata naspram faktora opterećenja, originalna implementacija

Grafovi su stepenastog oblika jer se FPR mjeri za svakih 10000 umetanja. Umjetno generirani podaci imaju viši FPR jer su ravnomjernije raspoređeni u tablici te se stvara više kolizija.



Slika 14: Propusnost umetanja naspram faktora opterećenja, naša implementacija



Slika 15: Propusnost umetanja naspram faktora opterećenja, originalna implementacija

Naša implementacija postiže visoku početnu propusnost, ali neravnomjerno pada. Originalna implementacija ima manju početnu vrijednost propusnosti, ali stabilnije pada. Za $k = 10$ jako brzo pada zbog puno neuspješnih umetanja duplikata.

3. Zaključak

Vacuum filter je vrsta AMQ (engl. *approximate membership query*) podatkovnih struktura koji je brži i učinkoviti od ostalih AMQ struktura. Uspješno je implementiran za bilježenje i traženje slučajnih podnizova u genomu bakterije *E. coli*. Implementirane su metode umetanja, brisanja i pretraživanja. Testiranja su provedena na genomu *E. coli* s različitim vrijednostima k (10, 20, 50, 100 i 200) i na umjetno generiranim podacima. Implementacija VF-a pokazuje da je faktor opterećenja oko 95%, što odgovara originalnoj implementaciji, udio lažno pozitivnih rezultata je vrlo malen za 16-bitne otiske prstiju i propusnosti su u skladu (čak i bolje) od originalne implementacije. Glavna razlika se pojavila kod iskorištenja memorije gdje naš VF koristi više memorije zbog neimplementiranih polu-sortiranih podataka što povećava potrošnju oko 23% (pod pretpostavkom da koristimo 16 bitne otiske, a kad bi koristili polu-sortirane otiske mogli bi se koristiti 13 bitni otisci).

4. Literatura

- [1] Wang, Minmei, et al. "Vacuum Filters: More Space-Efficient and Faster Replacement for Bloom and Cuckoo Filters." *Proceedings of the VLDB Endowment*, vol. 13, no. 2, Oct. 2019, pp. 197–210. DOI.org (Crossref), <https://doi.org/10.14778/3364324.3364333>.
- [2] Fan, Bin, et al. "Cuckoo Filter: Practically Better Than Bloom." *Proceedings of the 10th ACM International on Conference on Emerging Networking Experiments and Technologies* [Sydney Australia], 2014, pp. 75–88. DOI.org (Crossref), <https://doi.org/10.1145/2674005.267499>

